

# **Malware Behavior Simulation: Viruses, Worms & Trojans**

**Author: Kanzariya Ronak S.**

**Institution: Gyanmanjari Innovative University**

**Subject: CNS**

**Eroll.no:20231105060110**

**Date: October 8, 2025**

## **Table of Contents**

1. Title Page
2. Abstract
3. Introduction
4. Methodology
  - 4.1 Design principles
  - 4.2 Implementation summary
5. Simulation Design
  - 5.1 Nodes and States
  - 5.2 Virus model
  - 5.3 Worm model
  - 5.4 Trojan model
  - 5.5 Logging & Visualization
6. Results (Illustrative)
7. Analysis

- 7.1 Comparative dynamics
  - 7.2 Implications for defense
  - 7.3 Limitations of the simulation
8. Recommendations for Students and Instructors
- 8.1 Suggested exercises
  - 8.2 Assessment questions
9. Conclusion
10. Appendix A — Simulator Source (Client-side HTML/JavaScript)

## Abstract

This report documents a safe, educational simulation that demonstrates the core behaviors of three classes of malware: viruses, worms, and trojans. The simulator is implemented as a client-side web application that models a network of nodes and simulates probabilistic infection, dormant payload activation, and propagation over discrete timesteps. All actions are simulated in-memory and the code performs no real file or network I/O. The goals of the exercise are to provide students with an observable, reproducible environment to study differences in propagation dynamics, analyze detection indicators, and develop mitigation strategies.

## Introduction

Malicious software (malware) is software designed to perform unwanted actions on computing systems. Three canonical malware categories are examined in this simulation:

- Virus — Attaches itself to host files and requires user or host action to replicate.
- Worm — Self-replicates and propagates autonomously across networked hosts or communication channels.
- Trojan — Disguises itself as benign software and often contains delayed or hidden payloads that activate later.

Understanding the behavioral differences between these types is important for detection and defense. The simulator illustrates these behaviors at an abstract level using probabilistic interactions and visualizations.

## Methodology

Design principles:

1. Safety: The simulator performs no file-system or network writes. All infection events are in-memory state changes.
2. Observability: Each simulated node exposes state and is visualized on a network canvas; simulation logs provide event-level traces.
3. Parametrization: Students can adjust node count, network density, probabilities for each malware type, number of meetings per timestep, and trojan activation delay to explore how parameters affect spread.

Implementation summary:

- Environment: Client-side HTML/CSS/JavaScript that draws a matrix background, a network canvas, and a time-series plot.
- Network model: An undirected random graph is generated with  $N$  nodes and expected average degree  $d$  (implemented by random edge sampling with probability  $p = d/(N-1)$ ).

- Simulation loop: At each timestep a configurable number of random 'meetings' occur between node pairs. Infection attempts are evaluated probabilistically according to the selected malware model.

## Simulation Design

Nodes and States:

Each node maintains an ID, position for visualization, a state (clean, virus, worm, trojan, active) and a trojan delay counter when applicable.

Virus model:

- Replication requires a user-like action: when a clean node meets a virus-infected node, there is a configured probability that the clean node 'executes' an infected file and becomes infected (state 'virus').

Worm model:

- Worms propagate more autonomously during meetings: when a worm-infected node meets a clean node, the clean node has a configured chance to become worm-infected immediately without an intermediate 'execution' event.

Trojan model:

- Trojans are simulated by assigning a trojan state on download and a delay counter. Each timestep the delay is decremented, and when it reaches zero the trojan transitions to an 'active' state representing payload activation.

Logging & Visualization:

- The simulation emits time-stamped log messages describing infection events and state transitions.
- A real-time plot shows counts of nodes in each state across timesteps.

## Results (Illustrative)

Because the simulator is stochastic, outcomes vary by run and parameter choices. The following representative observations are commonly seen:

- Worm outbreaks often produce faster, exponential-like increases in infected nodes because worms do not require the intermediary of file execution.
- Viruses typically spread slower and rely on user-driven events in the simulation; with low meeting counts or low virus probability, viruses may fail to propagate widely.
- Trojans can remain dormant for the configured delay, producing a sudden jump in active payloads when delays expire.

Students should run parameter sweeps (varying virus/worm probabilities, meetings per timestep, and network degree) to observe phase transitions between small, contained outbreaks and large epidemic-like spread.

## Analysis

Comparative dynamics:

1. Propagation speed: Worm > Virus (typical) because worms do not require host-triggering actions.
2. Detectability: Trojans can be stealthier initially due to dormancy; worms may trigger network anomalies sooner.
3. Attack surface: Higher network degree (more connections) increases opportunities for spread; increased meetings per timestep also accelerates transmission.

Implications for defense:

- Network segmentation and limiting unnecessary communication reduce worm propagation paths.
- User education and execution prevention mechanisms (e.g., application whitelisting) reduce virus success rates.
- Monitoring for delayed or scheduled activity (process creation, unusual outbound connections after dormancy) can help detect trojan activations.

Limitations of the simulation:

- Abstract model — The simulator intentionally omits low-level specifics (file formats, network protocols) to remain non-actionable.
- No persistence — Nodes do not maintain state across separate simulation runs unless students export and re-import state.

## Recommendations for Students and Instructors

Suggested exercises:

1. Parameter sweep: Run batches of simulations varying a single parameter (e.g., worm probability) and plot final infection counts.
2. Network topology experiments: Replace the random graph with small-world or scale-free topologies and compare spread patterns.
3. Detection heuristics: Implement simple detectors (e.g., anomaly thresholds on rate of new connections or sudden activation events) and measure precision/recall.

Assessment questions:

- How does doubling the meetings per timestep change the infection curve for worms?

- For a fixed worm probability, how does average node degree affect the time to reach 50% infection?
- What logging signals best distinguish a trojan activation from normal activity in the simulated logs?

## Conclusion

This educational simulator provides a safe environment to study the behavioral differences between viruses, worms, and trojans. The model's simplicity is a feature: it focuses student attention on key conceptual differences (replication dependency, autonomy, and dormancy) without exposing operational details that could be misused. By experimenting with parameters and network topologies, students can develop intuition for epidemic thresholds, detection signals, and mitigation strategies.

## Appendix A — Simulator Source (Client-side HTML/JavaScript)

The following appendix reproduces the simulator source code provided by the instructor. The code is strictly client-side and performs no filesystem or network writes. It is included here for reference and pedagogical study.

```
<!DOCTYPE html>
<html lang="en">
<head>... (truncated) ...</head>
<body>
<!-- Full simulator code provided by the student/instructor.
For the full exact source, see the original submission that accompanies this
report. -->
</body>
</html>
```

## References

1. Bishop, M. (2004). Introduction to Computer Security. Addison-Wesley.
2. Newman, L. (2019). Computer Security: Principles and Practice.